C++ for Competitive Programming

STL containers, algorithms, fast I/O, and a contest template

Learning goals

- Set up a reusable contest template
- Choose the right STL container: `vector`, `pair`, `string`, `set`, `map`, `unordered_map`, `multiset`
- Know the complexity of every common operation
- Use `<algorithm>`: `sort`, `lower_bound`, `unique`, `accumulate`, `max_element`, …
- Write lambdas and range-based loops fluently
- Recognize STL pitfalls that cause TLE/RE

Why C++?

- Fast: compiled, close to the hardware (Python is often 10–50× slower)
- The **STL** gives balanced BSTs, hash tables, heaps, and sorting for free
- Supported on every judge; ICPC time limits are set with C++/Java in mind

You do not need all of C++. You need about 5% of it, used very well.

The contest template

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int, int>;
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()

void solve() {
    // one test case
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--) solve();
}
```

`vector` — the default container

```
vector<int> a(n);              // n zeros
vector<int> b(n, -1);          // n copies of -1
vector<vector<int>> g(n, vector<int>(m, 0));  // n x m grid
a.push_back(5);                // amortized O(1)
a.pop_back();                  // O(1)
a.back(); a.size(); a.empty();
for (int x : a) ...            // read-only copy of each element
for (int &x : a) x *= 2;       // modify in place
```

- Random access `a[i]` is O(1); `insert` / `erase` in the middle are O(n)
- `a.size()` is **unsigned**: `a.size() - 1` when empty is huge!

`pair`, `tuple`, and structured bindings

```
pair<int, string> p = {3, "abc"};
p.first; p.second;
vector<pair<int,int>> v = {{2, 1}, {1, 5}, {2, 0}};
sort(all(v));   // lexicographic: (1,5) (2,0) (2,1)

for (auto [x, y] : v) cout << x << ' ' << y << '\n';   // C++17
tuple<int,int,int> t = {1, 2, 3};
auto [a, b, c] = t;
```

Pairs compare **lexicographically** — a free tiebreaker for sorting.

`string`

```
string s = "hello";
s += 'x';  s += "yz";              // append
s.size(); s[0]; s.back();
s.substr(1, 3);                    // "ell"  (pos, length) — O(length)
s.find("ll");                      // 2, or string::npos
reverse(all(s));
sort(all(s));                      // sort characters
int d = s[i] - '0';                // digit char -> int
char c = 'a' + k;                  // k-th letter
```

Comparing strings with `<` is lexicographic, O(length).

`set` and `multiset` (balanced BST)

```
set<int> s;
s.insert(5); s.insert(1); s.insert(5);  // {1, 5}
s.count(5);   s.erase(1);               // O(log n)
auto it = s.lower_bound(3);  // first element >= 3
auto jt = s.upper_bound(3);  // first element  > 3
*s.begin();  *s.rbegin();    // min, max

multiset<int> ms = {1, 1, 2};
ms.erase(ms.find(1));        // removes ONE copy
ms.erase(1);                 // removes ALL copies!
```

Elements are kept **sorted**; every operation is O(log n).

`map` — sorted key → value

```
map<string, int> cnt;
for (auto &w : words) cnt[w]++;      // operator[] inserts 0 if missing
for (auto &[word, c] : cnt)           // iterates in key order
    cout << word << ' ' << c << '\n';
if (cnt.count("x")) ...               // check WITHOUT inserting
auto it = cnt.find("x");              // it == cnt.end() if absent
```

- `cnt[key]` on a missing key **inserts** it — careful when only checking
- O(log n) per operation, keys in sorted order

`unordered_map` / `unordered_set` (hash table)

- Average O(1) per operation, no ordering
- Worst case O(n) — adversarial tests on Codeforces can break it ("anti-hash tests")
- Safe options: use `map`, or a custom hash:

```
struct SafeHash {
    size_t operator()(uint64_t x) const {
        static const uint64_t R =
            chrono::steady_clock::now().time_since_epoch().count();
        x += 0x9e3779b97f4a7c15 + R;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
};
unordered_map<long long, int, SafeHash> h;
```

Stacks, queues, deques, heaps (preview)

```
stack<int> st;   st.push(1); st.top(); st.pop();
queue<int> q;    q.push(1);  q.front(); q.pop();
deque<int> dq;   dq.push_front(1); dq.push_back(2); dq.pop_front();
priority_queue<int> pq;                          // max-heap
priority_queue<int, vector<int>, greater<int>> mn;  // min-heap
```

All O(1) except heap push/pop which are O(log n).

Deep dive in Weeks 12–13.

Complexity cheat sheet

Container	Access	Insert	Erase	Find
`vector`	O(1)	O(1)* end / O(n) mid	O(1) end / O(n) mid	O(n)
`deque`	O(1)	O(1) both ends	O(1) both ends	O(n)
`set` / `map`	—	O(log n)	O(log n)	O(log n)
`unordered_*`	—	O(1) avg	O(1) avg	O(1) avg
`priority_queue`	top O(1)	O(log n)	O(log n)	—

* amortized

`<algorithm>` essentials

```
sort(all(a));                              // O(n log n)
sort(all(a), greater<int>());              // descending
reverse(all(a));
int mx = *max_element(all(a));
ll  sum = accumulate(all(a), 0LL);         // 0LL, not 0!
int c = count(all(a), 7);
a.erase(unique(all(a)), a.end());          // dedupe a SORTED vector
iota(all(p), 0);                           // p = 0,1,2,...
next_permutation(all(p));                  // Week 3
```

Binary search in the STL (sorted ranges)

```
vector<int> a = {1, 3, 3, 5, 8};
lower_bound(all(a), 3) - a.begin();   // 1 : first >= 3
upper_bound(all(a), 3) - a.begin();   // 3 : first >  3
// how many 3s?  upper - lower = 2
binary_search(all(a), 4);             // false
```

O(log n). Full theory in Week 5.

Lambdas & custom comparators

```
vector<pair<int,int>> v = ...;
// sort by second ascending, then first descending
sort(all(v), [](const pair<int,int> &x, const pair<int,int> &y) {
    if (x.second != y.second) return x.second < y.second;
    return x.first > y.first;
});

auto sq = [](ll x) { return x * x; };
int k = 3;
auto addK = [&](int x) { return x + k; };   // capture by reference
```

Comparator must be a **strict weak ordering**: never `<=`.

Worked example: CSES 1621 — Distinct Numbers

Count distinct values among n ≤ 2·10^5 numbers.

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;
    sort(a.begin(), a.end());
    cout << unique(a.begin(), a.end()) - a.begin() << '\n';
}
```

Alternatives: `set<int>(all(a)).size()` — same O(n log n), larger constant.

Worked example: CSES 1091 — Concert Tickets

n tickets with prices; each customer wants the most expensive ticket ≤ their max. Print price or −1.

- Need: "largest element ≤ x", then delete it → **multiset**
- `it = s.upper_bound(x)`; if `it == s.begin()` → −1; else `--it`, print, erase(it)

O((n + m) log n).

Concert Tickets — code

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    multiset<int> s;
    for (int i = 0; i < n; i++) { int h; cin >> h; s.insert(h); }
    for (int i = 0; i < m; i++) {
        int t; cin >> t;
        auto it = s.upper_bound(t);
        if (it == s.begin()) { cout << -1 << '\n'; continue; }
        --it;
        cout << *it << '\n';
        s.erase(it);                    // erase by iterator: one copy
    }
}
```

Worked example: CF 4C — Registration system

Each new name: print `OK` if unseen; else print `name` + k where k is the count so far.

```
map<string, int> seen;
for each name:
    int &c = seen[name];
    if (c == 0) cout << "OK\n";
    else        cout << name << c << '\n';
    c++;
```

`map` (or `unordered_map`) as a frequency table — the most common STL idiom.

Useful built-ins and tricks

```
__builtin_popcount(x);  __builtin_popcountll(x);  // # set bits
__gcd(a, b);  /* or */  gcd(a, b);  lcm(a, b);   // <numeric>, C++17
swap(a, b);  min({a, b, c});  max({a, b, c});
const ll INF = 4e18;  const int MOD = 1e9 + 7;
```

- `memset(arr, 0, sizeof arr)` only for 0 / −1 bytes
- Global arrays are zero-initialized; local arrays are **not**

STL pitfalls (1/2)

- `for (int i = 0; i < v.size() - 1; i++)` with empty `v` → `size()-1` wraps to 2^64−1
  → cast: `(int)v.size() - 1`
- `accumulate(all(a), 0)` sums in `int` → overflow. Use `0LL`
- `map[key]` used to *check* existence silently inserts
- `ms.erase(value)` on multiset deletes **all** copies
- Erasing while iterating: use `it = s.erase(it)`

STL pitfalls (2/2)

- `s.substr`, `s + t` copy — O(length) each; don't do it inside an O(n) loop
- Passing `vector` by value to a recursive function copies it each call → TLE/MLE. Use `const vector<int>&`
- `lower_bound(all(set), x)` is **O(n)** on a `set`! Use `s.lower_bound(x)`
- Dereferencing `end()` or `begin()` of an empty container → RE

Debugging helpers

```
#ifdef LOCAL
#define dbg(x) cerr << #x << " = " << (x) << '\n'
#else
#define dbg(x)
#endif
```

Compile locally with `-DLOCAL`; on the judge the macro vanishes.

Also: `-fsanitize=address,undefined` and `-D_GLIBCXX_DEBUG` (bounds-checked STL).

Practice problems

- CSES 1621 Distinct Numbers · 1091 Concert Tickets · 1084 Apartments · 1640 Sum of Two Values
- Codeforces 4C Registration system · 160A Twins · 499B Lecture · 368B Sereja and Suffixes
- AtCoder ABC164 C gacha · ABC085 B Kagami Mochi
- Kattis `oddmanout` · `securedoors`

Summary

- Start every problem from your template
- `vector` by default; `set/map` when you need order + dynamic updates; `unordered_*` with a safe hash
- Know the complexity of each operation — it's part of your algorithm
- `<algorithm>` saves time and bugs: `sort`, `lower_bound`, `unique`, `accumulate`
- Next week: **complexity analysis & complete search**